1. Java Basics

- What are the main features of Java?
- Explain the difference between JDK, JRE, and JVM.
- What is the difference between == and equals() in Java?
- What is the difference between final, finally, and finalize()?
- What are access modifiers in Java?
- What is the difference between throw and throws?
- What is the difference between checked and unchecked exceptions?
- What is garbage collection in Java?

2. Object-Oriented Programming (OOP)

- Explain the four pillars of OOP (Encapsulation, Inheritance, Polymorphism, Abstraction).
- What is method overloading and method overriding?
- Can you override a static method in Java?
- What is the difference between an abstract class and an interface?
- What is multiple inheritance? Does Java support it?
- What is the purpose of the super keyword?
- What are constructors and what are their types?

---

3. Collections Framework

- What is the difference between List, Set, and Map?
- Difference between ArrayList and LinkedList?
- Difference between HashMap and TreeMap?
- What is the difference between HashSet and LinkedHashSet?
- What is the difference between HashMap and ConcurrentHashMap?
- How does HashMap work internally (hashing and buckets)?
- What is a fail-fast iterator?

---

4. Multithreading & Concurrency

- What is a thread in Java?
- Difference between Runnable and Thread class?
- What is the purpose of the synchronized keyword?
- Difference between synchronized and Lock interface?
- What are volatile and transient keywords?
- What is ThreadLocal in Java?
- Explain ExecutorService and thread pools.
- What is a deadlock? How can you avoid it?

---

5. Java 8+ Features

- What are lambda expressions?
- What is the Stream API?
- Difference between map() and flatMap()?
- What are functional interfaces?
- What is Optional and how do you use it?
- Difference between forEach() and map()?
- Explain the concept of method references.

---

6. Exception Handling

- How is exception handling implemented in Java?
- What is the difference between checked and unchecked exceptions?
- Can you create your own exception class?
- What happens if you don't catch an exception?

Q1. What are the main features of Java?

Answer:

- Object-Oriented
- Platform Independent (via JVM)
- Robust and Secure
- Multithreaded
- High Performance (with JIT compiler)
- Automatic Memory Management (Garbage Collection)

Q2. Difference between JDK, JRE, and JVM?

Component	Description
JDK (Java Development Kit)	Contains tools for development (compiler, debugger, JRE, etc.)
JRE (Java Runtime Environment)	Provides libraries and JVM to run Java applications
JVM (Java Virtual Machine)	Executes bytecode and provides platform independence

---

Q3. Difference between == and equals()?

- == compares **reference** (memory address).
- equals() compares **content** (logical equality).

Example:

```
String s1 = new String("Java");
String s2 = new String("Java");
```

```
System.out.println(s1 == s2);      // false
System.out.println(s1.equals(s2)); // true
```

Q4. Difference between final, finally, and finalize()?

Keyword	Purpose
final	Used with variable (constant), method (cannot override), or class (cannot extend).
finally	Block that executes after try-catch (used for cleanup).
finalize()	Method called by Garbage Collector before object destruction (deprecated).

Q5. What are access modifiers in Java?

Modifier	Access Level
private	Within same class
default	Within same package
protected	Within package + subclasses
public	Accessible everywhere

Q6. Difference between throw and throws?

Keyword	Purpose
throw	Used to explicitly throw an exception.
throws	Declares exceptions a method might throw.

Example:

```
void m1() throws IOException {
   throw new IOException("Error occurred");
}
```

Q7. Difference between checked and unchecked exceptions?

- **Checked:** Checked at compile-time (e.g., IOException, SQLException).
- **Unchecked:** Checked at runtime (e.g., NullPointerException, ArithmeticException).

2. Object-Oriented Programming (OOP)

Q8. Four pillars of OOP?

1. **Encapsulation** – Data hiding using private fields and public getters/setters.
2. **Inheritance** – Reuse behavior from a parent class using extends.
3. **Polymorphism** – Same method behaves differently (compile-time & runtime).
4. **Abstraction** – Hiding implementation details using abstract classes/interfaces.

Q9. What is method overloading and method overriding?

- **Overloading:** Same method name, different parameters (compile-time polymorphism).

- **Overriding:** Same method signature, subclass changes behavior (runtime polymorphism).

Example:

```
class A {
   void display(int a) { }
}
class B extends A {
   @Override
   void display(int a) { }  // overriding
}
```

Q10. Can we override a static method in Java?

No.

Static methods belong to the class, not the object.

They can be hidden, but not overridden.

Q11. Difference between abstract class and interface?

Feature	Abstract Class	Interface
Methods	Can have abstract & concrete methods	All methods are abstract (Java 7), default/static allowed (Java 8+)
Variables	Can have instance variables	Only constants (public static final)
Inheritance	Single inheritance	Multiple inheritance supported
Use case	Common behavior	Contract definition

Q12. Does Java support multiple inheritance?

No, Java does not support multiple inheritance with classes (to avoid ambiguity),

but it **does** support multiple inheritance through interfaces.

Q13. What is the super keyword used for?

- Refers to the **immediate parent class object**.

  Used to:

1. Call parent constructor (super()).
2. Access parent methods/variables when overridden.

Q14. What are constructors?

- Special methods used to initialize objects.
- **Types:**
  - Default constructor (no parameters)
  - Parameterized constructor

- Copy constructor (manually defined)

Example:

```
class Student {
   String name;
   Student(String n) { name = n; }
}
```

Q1. What is the Java Collections Framework (JCF)?

Answer:

The Java Collections Framework is a **set of classes and interfaces** that provide a standard way to store, manipulate, and access groups of objects efficiently.

Main interfaces:

- List – Ordered collection, allows duplicates.
- Set – No duplicates, unordered.
- Map – Key-value pairs, no duplicate keys.
- Queue – FIFO order for processing elements.

Q2. Difference between List, Set, and Map

Feature	List	Set	Map
Duplicates	Allowed	Not allowed	Keys – Not allowed, Values – Allowed
Order	Maintained	Not guaranteed	Key order depends on implementation
Access	By index	By object	By key
Examples	ArrayList, LinkedList	HashSet, TreeSet	HashMap, TreeMap

---

Q3. Difference between ArrayList and LinkedList

Feature	ArrayList	LinkedList
Storage	Dynamic array	Doubly linked list
Access time	Fast for get/set (O(1))	Slower for get/set (O(n))
Insertion/Deletion	Slow in middle (shift elements)	Fast (no shifting needed)
Use case	Frequent read	Frequent insert/delete

Q4. Difference between HashMap and TreeMap

Feature	HashMap	TreeMap
Order	Unordered	Sorted by keys
Performance	Faster (O(1))	Slower (O(log n))
Null keys	Allows one null key	Doesn't allow null keys
Use case	Fast lookup	Sorted map needs

Q5. Difference between HashSet and LinkedHashSet

Feature	HashSet	LinkedHashSet
Order	Unordered	Maintains insertion order
Performance	Slightly faster	Slightly slower
Use case	Unordered unique elements	Maintain order of insertion

---

Q6. How does a HashMap work internally?

Answer:

1. A HashMap uses an **array of buckets** to store key-value pairs.
2. Each key's hashCode() determines its bucket index.
3. If multiple keys map to the same index → **collision** occurs.
4. Collisions are resolved using a **linked list** (Java 7) or **balanced tree** (Java 8+).
5. Lookup time is **O(1)** on average and **O(log n)** in worst case.

Q7. What is a fail-fast iterator?

Answer:

- If a collection is modified while iterating (except via iterator's own methods), it throws a **ConcurrentModificationException**.
- Example: ArrayList, HashMap iterators are fail-fast.
- Concurrent collections like ConcurrentHashMap are **fail-safe** (no exception).

Q8. Difference between HashMap and ConcurrentHashMap

Feature	HashMap	ConcurrentHashMap
Thread safety	Not thread-safe	Thread-safe
Performance	Faster (single-thread)	Slightly slower (multi-threaded)
Null keys	Allows one null key	Does NOT allow null key/value
Behavior on concurrency	May throw ConcurrentModificationException	Safe concurrent access

---

Q9. What is the difference between Iterator and ListIterator?

Feature	Iterator	ListIterator
Traversal	Only forward	Forward and backward
Applicable for	All collections	Only for List
Add elements	Not possible	Possible
Modify elements	Possible via set()	Possible via set()

Q10. What are comparable and comparator interfaces?

Interface	Purpose
Comparable	Defines natural ordering within the class (compareTo()).

Interface	Purpose
Comparator	Defines custom ordering externally (compare()).

Example:

```
class Student implements Comparable<Student> {
   int age;
   public int compareTo(Student s) {
      return this.age - s.age;
   }
}
```

Custom comparator:

```
Comparator<Student> nameComparator = (a, b) -> a.name.compareTo(b.name);
```

Q11. Difference between Vector and ArrayList

Feature	Vector	ArrayList
Synchronization	Synchronized (thread-safe)	Not synchronized
Performance	Slower	Faster
Growth	Doubles size	Grows by 50%
Legacy	Yes (old)	Modern collection

Q12. What is the difference between Enumeration, Iterator, and ListIterator?

Type	Direction	Modify allowed	Legacy
Enumeration	Only forward	No	Yes
Iterator	Forward only	Yes	No
ListIterator	Both directions	Yes	No

Q13. Difference between Collection and Collections

Term	Type	Description
Collection	Interface	Root interface of List, Set, Queue
Collections	Utility class	Provides static methods like sort(), reverse(), shuffle(), etc.

Q14. What are synchronized collections?

Answer:

- Thread-safe wrappers over standard collections using Collections.synchronizedList() or synchronizedMap().

  Example:

```
List<String> list = Collections.synchronizedList(new ArrayList<>());
```

Q15. What is the difference between peek(), poll(), and remove() in Queue?

Method	Action	Behavior
peek()	Retrieves head	Returns null if empty
poll()	Retrieves & removes head	Returns null if empty
remove()	Retrieves & removes head	Throws exception if empty

Perfect 👍 — let's continue with **Part 4: Multithreading & Concurrency in Java** — one of the most asked areas in interviews.

⚙️ **Core Java – Multithreading & Concurrency (with Solutions)**

Q1. What is a thread in Java?

Answer:

A **thread** is the smallest unit of a process that can run independently.

In Java, every program runs on at least one thread — the **main thread**.

Threads allow **concurrent execution** of multiple parts of a program for better performance.

Q2. How can you create a thread in Java?

There are **two main ways:**

1️⃣ **By extending the Thread class:**

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running...");
    }
}
public class Demo {
    public static void main(String[] args) {
        new MyThread().start();  // start() invokes run() in new thread
    }
}
```

2️⃣ **By implementing the Runnable interface:**

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable thread running...");
    }
}
public class Demo {
    public static void main(String[] args) {
        Thread t = new Thread(new MyRunnable());
```

```
        t.start();
    }
}
```

✅ **Best practice:** use Runnable — because Java supports multiple interfaces, not multiple inheritance.

Q3. Difference between Runnable and Thread class

Feature	Runnable	Thread
Inheritance	Interface	Class
Multiple inheritance	Allowed	Not allowed
Reusability	Can be shared among threads	Not reusable
Preferred	✔ Yes	❌ Less preferred

Q4. What is the purpose of the synchronized keyword?

Answer:

Used to ensure that **only one thread** can access a block or method at a time — avoids **race conditions**.

Example:

```
synchronized void display() {
    // critical section
}
or
synchronized(this) {
    // synchronized block
}
```

Q5. Difference between synchronized and Lock interface

Feature	synchronized	Lock
Type	Keyword	Interface (java.util.concurrent.locks)
Unlocking	Automatic	Manual (lock() / unlock())
Try lock	Not possible	Possible with tryLock()
Fairness	No fairness guarantee	Can ensure fair order
Performance	Simple, limited control	More flexible and powerful

---

Q6. What is the volatile keyword?

Answer:

volatile ensures that a variable's value is **always read from main memory**, not from a thread's local cache — used to achieve **visibility** between threads.

Example:

```
volatile boolean flag = true;
```

Q7. What is the transient keyword?

Answer:

Used to **exclude variables from serialization**.

When an object is serialized, transient fields are **not saved**.

Q8. What is a ThreadLocal variable?

Answer:

A variable whose value is **unique to each thread** — threads don't share it.

Example:

```
ThreadLocal<Integer> counter = ThreadLocal.withInitial(() -> 0);
counter.set(counter.get() + 1);
```

Each thread will have its **own copy** of counter.

Q9. What is ExecutorService and why is it used?

Answer:

ExecutorService manages thread creation and reuse.

Instead of manually creating threads, you submit tasks to a **thread pool**.

Example:

```
ExecutorService executor = Executors.newFixedThreadPool(3);
executor.submit(() -> System.out.println("Task executed"));
executor.shutdown();
```

✅ Benefits:

- Efficient thread reuse
- Better performance
- Simplified thread management

Q10. What are Callable and Future in Java?

Answer:

- Callable<T>: Similar to Runnable but **returns a value** and **can throw exceptions**.
- Future<T>: Represents the **result of an asynchronous computation**.

Example:

```
Callable<Integer> task = () -> 10 + 20;
Future<Integer> result = executor.submit(task);
System.out.println(result.get());  // waits for completion
```

Q11. What is a deadlock?

Answer:

A situation where two or more threads are waiting for each other's locks indefinitely.

Example:

```
synchronized(lock1) {
  synchronized(lock2) { }
}
synchronized(lock2) {
  synchronized(lock1) { }
}
```

✅ **Avoid deadlocks by:**

- Acquiring locks in a consistent order
- Using tryLock()
- Keeping synchronized sections small

Q12. What is a race condition?

Answer:

Occurs when multiple threads access and modify shared data **simultaneously**, leading to inconsistent results.

Solution: Use synchronization or atomic variables.

Q13. What is the difference between process and thread?

Feature	Process	Thread
Memory	Separate memory	Shared memory
Communication	Slow (IPC)	Fast
Creation	Heavy	Lightweight
Failure impact	Independent	Can crash process if not handled

Q14. What is the difference between wait(), notify(), and notifyAll()?

Method	Description
wait()	Makes current thread wait until another thread invokes notify() or notifyAll()
notify()	Wakes up one waiting thread
notifyAll()	Wakes up all waiting threads

Note: Must be called from within a synchronized block.

Q15. What are atomic classes in Java?

Answer:

Atomic classes (like AtomicInteger, AtomicBoolean, etc.) from java.util.concurrent.atomic

provide **lock-free**, **thread-safe** operations using low-level **CAS (Compare-And-Swap)**.

Example:

AtomicInteger counter = new AtomicInteger(0);

counter.incrementAndGet();

Q16. What is the difference between sleep() and wait()?

Feature	sleep()	wait()
Belongs to	Thread class	Object class
Lock release	Does NOT release lock	Releases lock
Use case	Pause execution	Inter-thread communication

Q17. What is the difference between concurrency and parallelism?

Concept	Description
Concurrency	Multiple tasks make progress in overlapping time (not necessarily at once).
Parallelism	Multiple tasks run **simultaneously** on multiple cores.

Q18. What is the difference between CyclicBarrier and CountDownLatch?

Feature	CountDownLatch	CyclicBarrier
Resettable	No	Yes
Usage	Wait for other threads to finish	Wait for threads to reach a common barrier point

---

Q19. What is the Fork/Join framework?

Answer:

Used to break tasks into smaller **subtasks** (fork), process them in parallel, and **combine results** (join).

Best for **divide-and-conquer** problems.

Q20. What is the difference between Executor, ExecutorService, and ScheduledExecutorService?

Interface	Description
Executor	Basic interface for executing tasks
ExecutorService	Extends Executor, adds lifecycle management and task submission
ScheduledExecutorService	Supports task scheduling at fixed rate or delay

---

☕ **Core Java – Java 8+ Features (with Solutions)**

Q1. What are the key features introduced in Java 8?

Answer:

- **Lambda Expressions**
- **Functional Interfaces**
- **Stream API**
- **Default and Static Methods in Interfaces**
- **Optional Class**
- **Date and Time API (java.time)**
- **Method References**
- **Nashorn JavaScript Engine**

Q2. What are Lambda Expressions in Java?

Answer:

A **lambda expression** is a short way to represent a **function** (anonymous method).

They make code **concise** and **functional-style**.

Syntax:

(parameter) -> expression

Example:

```
List<Integer> list = Arrays.asList(1, 2, 3);
list.forEach(n -> System.out.println(n));
```

✅ Without lambda:

```
list.forEach(new Consumer<Integer>() {
   public void accept(Integer n) {
      System.out.println(n);
   }
});
```

Q3. What is a Functional Interface?

Answer:

An interface that has **only one abstract method**.

Can have multiple default or static methods.

Annotated with @FunctionalInterface.

Example:

```
@FunctionalInterface
interface Calculator {
   int add(int a, int b);
}
```

Used as a lambda:

```
Calculator c = (a, b) -> a + b;
```

✅ Common functional interfaces:

- Runnable, Callable
- Predicate<T>
- Function<T, R>
- Consumer<T>
- Supplier<T>

Q4. What is the Stream API?

Answer:

The **Stream API** is used to **process collections** (like filtering, mapping, reducing) in a **declarative and functional style**.

Example:

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> even = nums.stream()
                .filter(n -> n % 2 == 0)
                .collect(Collectors.toList());
System.out.println(even);  // [2, 4]
```

Q5. Difference between map() and flatMap()

Method	Purpose
map()	Transforms each element (1-to-1 mapping)
flatMap()	Flattens nested streams (1-to-many mapping)

Example:

```
List<List<Integer>> list = Arrays.asList(Arrays.asList(1,2), Arrays.asList(3,4));

List<Integer> result = list.stream()
                .flatMap(l -> l.stream())
                .collect(Collectors.toList());
// Output: [1,2,3,4]
```

Q6. What is Optional in Java 8?

Answer:

Optional is a **container class** used to handle **null values** gracefully and avoid NullPointerException.

Example:

```
Optional<String> name = Optional.ofNullable(null);
System.out.println(name.orElse("Default Name")); // Output: Default Name
```

Useful methods:

- isPresent()
- orElse()

- orElseGet()
- orElseThrow()
- ifPresent()

Q7. Difference between forEach() and map()

Method	Purpose	Returns
forEach()	Performs action (terminal operation)	void
map()	Transforms elements (intermediate operation)	New Stream

Example:

```
list.stream().map(n -> n * 2).forEach(System.out::println);
```

Q8. What are Method References in Java 8?

Answer:

A shorthand for lambda expressions that **refer to existing methods**.

Syntax:

```
ClassName::methodName
```

Examples:

```
// Static method reference
Function<Integer, String> f = String::valueOf;

// Instance method reference
Consumer<String> printer = System.out::println;

// Constructor reference
Supplier<List<String>> listSupplier = ArrayList::new;
```

Q9. What are Default and Static Methods in Interfaces?

Answer:

- **Default Method:** Defined in interface using default keyword. Allows backward compatibility.
- **Static Method:** Belongs to the interface itself.

Example:

```
interface Vehicle {
   default void start() { System.out.println("Vehicle started"); }
   static void horn() { System.out.println("Beep!"); }
}
```

Q10. What is the difference between Collection and Stream?

Feature	Collection	Stream
Nature	Stores data	Processes data
Modification	Mutable	Immutable
Traversal	Can traverse multiple times	Usually traversed once
Evaluation	Eager	Lazy
Example	List, Set	list.stream()

Q11. What are Intermediate and Terminal operations in Stream API?

Type	Examples	Description
Intermediate	filter(), map(), sorted()	Return a new Stream
Terminal	collect(), forEach(), count()	End the Stream pipeline

Example:

```
long count = list.stream()
        .filter(n -> n > 10)
        .count();
```

Q12. What is short-circuiting in Streams?

Answer:

When the Stream **terminates early** without processing all elements (e.g., findFirst(), anyMatch()).

Example:

```
list.stream()
  .filter(n -> n > 5)
  .findFirst()
  .ifPresent(System.out::println);
```

Q13. What is the new Date and Time API (java.time)?

Answer:

Introduced in Java 8 to replace Date and Calendar with **immutable**, thread-safe classes:

- LocalDate – date only
- LocalTime – time only
- LocalDateTime – date and time
- ZonedDateTime – with time zone
- Period / Duration – for differences

Example:

```
LocalDate today = LocalDate.now();
LocalDate future = today.plusDays(10);
System.out.println(future);
```

Q14. What is Stream Parallelism in Java 8?

Answer:

parallelStream() enables **parallel execution** of Stream operations across multiple cores.

Example:

```
list.parallelStream().forEach(System.out::println);
```

✅ Used for large data sets, but may not always be faster (depends on CPU and task size).

Q15. What are Collectors in Stream API?

Answer:

Collectors are **utility methods** used in terminal operations like collect().

Common examples:

```
// To List
List<Integer> list = stream.collect(Collectors.toList());

// To Set
Set<Integer> set = stream.collect(Collectors.toSet());

// Grouping
Map<String, List<Employee>> group = empList.stream()
   .collect(Collectors.groupingBy(Employee::getDept));

// Joining
String names = namesList.stream()
   .collect(Collectors.joining(", "));
```

Excellent 👏 — let's continue with **Part 6: Exception Handling & Memory Management in Java** — a favorite interview area because it tests both logic and understanding of the JVM.

⚙️ **Core Java – Exception Handling & Memory Management (with Solutions)**

🧩 **1. What is Exception Handling in Java?**

Answer:

Exception Handling is a mechanism to handle **runtime errors** so the program can continue executing instead of crashing.

Java uses **try-catch-finally** blocks and custom exception classes to handle errors gracefully.

Syntax:

```
try {
```

```
    // risky code
} catch (ExceptionType e) {
    // handling code
} finally {
    // cleanup code
}
```

🧩 2. What is the difference between Error and Exception?

Feature	Error	Exception
Type	Serious issues (JVM-related)	Recoverable problems (application-related)
Examples	OutOfMemoryError, StackOverflowError	IOException, NullPointerException
Handling	Cannot be handled	Can be handled using try-catch

🧩 3. Difference between Checked and Unchecked Exceptions

Type	Checked	Unchecked
Checked at	Compile-time	Runtime
Inheritance	Subclass of Exception	Subclass of RuntimeException
Example	IOException, SQLException	NullPointerException, ArithmeticException

Example:

```
// Checked
FileReader f = new FileReader("file.txt"); // must handle IOException


// Unchecked
int x = 10 / 0; // ArithmeticException
```

🧩 4. What is the difference between throw and throws?

Keyword	Used for	Example
throw	To explicitly throw an exception	throw new IOException("Error");
throws	To declare that a method may throw exceptions	void readFile() throws IOException

---

🧩 5. What is the finally block used for?

Answer:

Used to execute **cleanup code** (like closing files or database connections) regardless of whether an exception occurs or not.

Example:

```
try {
    int x = 10 / 0;
```

```
} catch (ArithmeticException e) {
    System.out.println("Error");
} finally {
    System.out.println("Finally executed");
}
```

🧩 6. Can finally block be skipped?

Yes — it will **not execute** if:

- The JVM exits (System.exit(0)).
- The thread is killed before reaching it.

🧩 7. Can we have multiple catch blocks?

Yes.

To handle different exception types separately.

Example:

```
try {
    int[] a = new int[2];
    a[5] = 10;
} catch (ArithmeticException e) {
    System.out.println("Arithmetic Exception");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Array Exception");
} catch (Exception e) {
    System.out.println("Parent Exception");
}
```

👉 Always put **specific exceptions first**, and **general ones last**.

🧩 8. What is “try-with-resources” in Java 7+?

Answer:

It automatically **closes resources** (like files or streams) without needing a finally block.

Example:

```
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
}
```

✅ The resource (br) is auto-closed when the try block ends.

🧩 9. Can we catch multiple exceptions in one catch block?

Yes, using the pipe (|) operator (Java 7+).

Example:

```
try {
   // code
} catch (IOException | SQLException e) {
   e.printStackTrace();
}
```

🧩 10. Can a try block exist without catch?

Yes — but it must be followed by **either catch or finally**.

Example:

```
try {
   int x = 10 / 0;
} finally {
   System.out.println("Cleanup");
}
```

🧩 11. What happens if an exception is not caught?

If an exception isn't caught:

- The JVM's **default exception handler** prints the stack trace.
- The program terminates abnormally.

🧩 12. Can we rethrow an exception?

Yes — we can throw the same exception again after handling it.

Example:

```
try {
   int x = 10 / 0;
} catch (ArithmeticException e) {
   System.out.println("Handled once");
   throw e; // rethrow
}
```

🧩 13. Can we create custom exceptions in Java?

Yes.

We can define our own exceptions by extending Exception or RuntimeException.

Example:

```
class InvalidAgeException extends Exception {
```

```
    public InvalidAgeException(String message) {
        super(message);
    }
}
```

Usage:

```
if (age < 18)
    throw new InvalidAgeException("Not eligible to vote");
```

🧩 14. What is Exception Chaining?

Answer:

It allows one exception to **wrap another**, preserving the original cause.

Example:

```
try {
    throw new IOException("File error");
} catch (IOException e) {
    throw new RuntimeException("Runtime exception", e);
}
```

🧩 15. What is the difference between getMessage() and printStackTrace()?

Method	Purpose
getMessage()	Returns description of the exception
printStackTrace()	Prints full stack trace (class, line number, etc.)

🧠 Memory Management in Java

🧩 16. What is Garbage Collection (GC)?

Answer:

Garbage Collection in Java **automatically removes unused objects** from memory to free space on the heap.

🧩 18. What is the difference between stack and heap memory?

Feature	Stack	Heap
Stores	Local variables, function calls	Objects, class data
Access	Fast	Slower
Memory	Automatically managed	Managed by GC
Scope	Per-thread	Shared among threads

🧩 19. What are memory leaks in Java?

Answer:

A **memory leak** occurs when objects are no longer needed but remain referenced, preventing GC from reclaiming them.

Example: Static collections storing large data without cleanup.

🧩 20. What is the finalize() method?

Answer:

- Called by the **Garbage Collector** before destroying an object.
- Used for resource cleanup (but **deprecated** in Java 9+).
- Modern approach: use **try-with-resources** or **AutoCloseable**.

🧩 21. How can you suggest JVM to run garbage collection?

System.gc(); // Suggests GC (not guaranteed)

But it's **just a request**, not a command — JVM decides when to collect.

🧩 22. How to prevent memory leaks in Java?

✅ **Best practices:**

- Close resources (I/O, DB connections)
- Use weak references for caches
- Avoid static references to large objects
- Use try-with-resources
- Profile memory with tools (e.g., VisualVM)

Perfect 👍 — let's continue with **Part 7: Spring Boot – Core Concepts, Annotations, and Internals (with Solutions)**.

This section covers **the most frequently asked Spring Boot interview questions**, especially for Java backend developers.

🌱 Spring Boot Interview Questions & Solutions

🧩 1. What is Spring Boot?

Answer:

Spring Boot is a framework built on top of the Spring Framework that simplifies application development by:

- Reducing boilerplate code.
- Providing **auto-configuration**.
- Including **embedded servers** (Tomcat, Jetty).
- Supporting **microservices** out of the box.

Example:

```
@SpringBootApplication
public class MyApp {
   public static void main(String[] args) {
      SpringApplication.run(MyApp.class, args);
   }
}
```

🧩 2. What are the advantages of using Spring Boot?

✅ Simplified configuration (no XML)

✅ Embedded servers (no need for WAR deployment)

✅ Auto dependency management via **starter POMs**

✅ Built-in **Actuator**, **DevTools**, and **Profiles**

✅ Ready for **microservices and REST APIs**

🧩 3. What does @SpringBootApplication do?

It is a **convenience annotation** that combines:

- @Configuration → defines beans
- @EnableAutoConfiguration → enables auto configuration
- @ComponentScan → scans the current package for components

Example:

```
@SpringBootApplication
public class DemoApp { }
```

🧩 4. What is Auto Configuration in Spring Boot?

Answer:

Spring Boot automatically configures your application based on the **dependencies** in the classpath.

For example:

- If spring-boot-starter-web is present → configures Tomcat + Spring MVC automatically.

You can disable it with:

@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})

🧩 5. What are Spring Boot Starters?

Answer:

Starters are **predefined dependency bundles** that simplify build configuration.

Examples:

Starter	Purpose
spring-boot-starter-web	For building REST APIs
spring-boot-starter-data-jpa	For JPA & Hibernate
spring-boot-starter-security	For authentication & authorization
spring-boot-starter-test	For testing

Example (in pom.xml):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

🧩 6. Explain Dependency Injection (DI) in Spring Boot

Answer:

DI allows you to **inject dependencies automatically** instead of manually creating objects using new.

Example:

```
@Service
public class UserService {
   @Autowired
   private UserRepository repo;
}
```

✅ Here, Spring automatically injects UserRepository into UserService.

🧩 7. What are the scopes of beans in Spring Boot?

Scope	Description
singleton	One instance per container (default)
prototype	New instance for every injection

request	One instance per HTTP request
session	One instance per HTTP session
application	One instance per web app context

Example:

```
@Scope("prototype")
@Component
public class MyBean { }
```

🧩 8. What is the difference between @Component, @Service, @Repository, and @Controller?

Annotation	Purpose
@Component	Generic bean
@Service	Business logic layer
@Repository	DAO layer, enables exception translation
@Controller	Web controller (returns views)
@RestController	REST controller (returns JSON/XML)

Example:

```
@RestController
public class UserController {
   @GetMapping("/hello")
   public String hello() {
      return "Hello Spring Boot!";
   }
}
```

🧩 9. What is the difference between @RestController and @Controller?

Feature	@Controller	@RestController
Return Type	View (HTML, JSP)	Data (JSON/XML)
Requires @ResponseBody	Yes	No (implicit)

🧩 10. What is application.properties (or application.yml)?

Answer:

Used to configure the Spring Boot application.

Example:

```
server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/test
spring.datasource.username=root
```

Or YAML version:

```
server:
  port: 8081
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/test
```

🧩 11. What is Spring Boot Actuator?

Answer:

A tool that provides **production-ready monitoring** and **metrics**.

Endpoints:

Endpoint	Description
/actuator/health	Health status
/actuator/info	App info
/actuator/metrics	System metrics
/actuator/env	Environment properties

Enable endpoints:

```
management.endpoints.web.exposure.include=*
```

🧩 12. What is the role of @Value and @ConfigurationProperties?

@Value — injects single property:

```
@Value("${server.port}")
private int port;
```

@ConfigurationProperties — binds multiple related properties:

```
@ConfigurationProperties(prefix = "app")
public class AppConfig {
  private String name;
  private String version;
}
```

```
app.name=DemoApp
app.version=1.0
```

🧩 13. How does Spring Boot handle profiles?

Profiles let you define **different configurations** for environments (dev, test, prod).

Example:

```
# application-dev.properties
server.port=8081


# application-prod.properties
```

server.port=80

Activate profile:

spring.profiles.active=dev

🧩 14. How to connect Spring Boot with a Database (JPA example)?

Step 1: Add dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
</dependency>
```

Step 2: Define application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/testdb
spring.datasource.username=root
spring.datasource.password=admin
spring.jpa.hibernate.ddl-auto=update
```

Step 3: Create Entity + Repository

```
@Entity
public class User {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
}
public interface UserRepository extends JpaRepository<User, Long> { }
```

🧩 15. What is CommandLineRunner?

Answer:

Used to run code after the Spring Boot app starts.

Example:

```
@Component
public class StartupRunner implements CommandLineRunner {
    @Override
    public void run(String... args) {
        System.out.println("Application started!");
```

```
    }
}
```

🧩 16. What is @Transactional used for?

Answer:

Manages transactions automatically for database operations.

Example:

```
@Service
public class UserService {
    @Transactional
    public void updateUser(User user) {
        // all DB operations in this method will roll back if an exception occurs
    }
}
```

🧩 17. How does Spring Boot differ from Spring MVC?

Feature	Spring MVC	Spring Boot
Setup	Manual (XML/Annotations)	Auto-configured
Deployment	Requires external server	Embedded Tomcat/Jetty
Dependency management	Manual	Starters (simplified)
Actuator	Not available	Built-in
Ideal for	Web apps	Microservices, REST APIs

🧩 18. What is the role of @EnableAutoConfiguration?

Answer:

It tells Spring Boot to automatically configure beans based on the **classpath dependencies**.

You can disable specific ones:

```
@EnableAutoConfiguration(exclude = {DataSourceAutoConfiguration.class})
```

🧩 19. How to create a custom exception handler in Spring Boot?

Example:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<String> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
```

```
    }
}
```

✅ Centralized exception handling for REST APIs.

🧩 20. What is SpringApplication.run() doing internally?

It performs:

1. Creates ApplicationContext
2. Loads beans
3. Starts embedded web server (Tomcat)
4. Runs CommandLineRunner and ApplicationRunner
5. Listens for events